13강

프로그래밍 언어론

서브프로그램 개요

컴퓨터과학과 김진욱교수

학습목차



1. 서브프로그램 개념
2. 매개변수와 인수 전달
3. 서브프로그램 관련 주제

01

서브프로그램 개념

1. 서브프로그램 개념

프로그램

- 컴퓨터가 실행할 명령어를 나열한 것
- 프로그램 실행을 통해 입력을 출력으로 바꿈

서브프로그램

- 독자적인 입력과 출력을 갖춘 프로그램 조각
  - 서브프로그램의 입력: 인수
  - 서브프로그램의 출력: 반환값

1. 서브프로그램 개념

서브프로그램의 특징

- 하나의 입구
- 여러 출구
- 기본 출구와 반환
  - 별도의 return문을 통해 출구 지정 가능
- 제어 흐름 이전
  - 호출자: 서브프로그램을 호출한 프로그램
  - 피호출자: 호출된 서브프로그램
- 제어 흐름 복귀

1. 서브프로그램 개념

서브프로그램의 분류

함수

- 명시적 반환값이 있는 서브프로그램

프로시저

- 명시적 반환값이 없는 서브프로그램

서브루틴

- 프로그래밍 언어 초기의 서브프로그램
- 인수 전달, 반환값 전달 등의 데이터 흐름이 명확하지 않음
- 입구가 여러 개일 수 있음

1. 서브프로그램 개념

❑ 서브프로그램 관련 용어

서브프로그램 정의

- 서브프로그램의 작동을 기술한 부분

서브프로그램 호출

- 서브프로그램이 수행될 것을 요구하는 부분

1. 서브프로그램 개념

서브프로그램 관련 용어

서브프로그램 헤더

- 서브프로그램의 이름, 종류, 매개변수 선언 등을 포함하는 부분
- 대개 서브프로그램 정의의 첫 줄이 됨

서브프로그램 본체

- 서브프로그램이 호출되었을 때 실행되는 부분
- 대개 서브프로그램 정의 중 헤더를 제외한 부분이 됨

1. 서브프로그램 개념

❑ 서브프로그램 관련 용어 - C

```
int sub(int i, int j)
{
  return i - j;
}

int main()
{
  int x = 3;
  x = sub(x * x, x);
  return x - 1;
}
```

1. 서브프로그램 개념

❑ 서브프로그램 관련 용어

매개변수 프로파일

- 서브프로그램 매개변수의 개수, 순서, 타입을 의미
- 시그니처(signature)라고도 함
- 이름이 같은 서브프로그램이 여러 개 오버로딩된 경우 서브프로그램 호출과 일치하는 서브프로그램을 찾는 데 사용
  - 오버로딩 결정, 시그니처 매칭

1. 서브프로그램 개념

❑ 서브프로그램 관련 용어

서브프로그램 프로토콜

- 매개변수 프로파일 + 반환 타입(함수의 경우)
- 서브프로그램 호출 시 타입 검사를 위해 필요
- 정적 타입 언어는 서브프로그램 호출 전에 서브프로그램 프로토콜이 명시될 것을 규정
  - 서브프로그램 선언

1. 서브프로그램 개념

서브프로그램 관련 용어 - C

```c
int sub(int, int);

int main()
{
  int x = 3;
  x = sub(x * x, x);
  return x - 1;
}

int sub(int i, int j)
{
  return i - j;
}
```

서브프로그램 선언:
서브프로그램 프로토콜만 명시

서브프로그램 정의:
서브프로그램 헤더와 본체 모두 명시

1. 서브프로그램 개념

형식인수와 실인수

- 서브프로그램 호출 시, 호출자의 실인수가 피호출자의 형식인수로 전달됨

형식인수 또는 형식매개변수

- 서브프로그램 정의에서 외부로부터 전달된 정보를 받기 위해 선언한 변수

실인수 또는 실매개변수

- 서브프로그램 호출 시 형식인수로 전달될 실제 값이나 주소

```
int sub(int i, int j)
{ ... }

int main()
{ ...
  x = sub(x * x, x);
```

1. 서브프로그램 개념

형식인수와 실인수의 대응

```
def sort(length, list):
  ...
```

위치에 의한 대응

- 가장 일반적
- C, Java, Pascal 등

```
sort(n, a)
```

키워드에 의한 대응

- 형식인수 이름에 의한 대응 방법
- 장점: 형식인수 순서를 기억하지 않아도 됨
- 단점: 형식인수 이름을 기억해야 함

```
sort(list=a, length=n)
```

- Ada, Fortran 90, Python 등은 두 방법 병용

1. 서브프로그램 개념

인수전달 고려 사항

기본인수

- 실인수가 주어지지 않을 경우 사용하는 값
- 형식인수의 기본값
- Ada, C++, Fortran 90, Python 등

```
void move(Robot r, int x = 0, int y = 0, int angle = 0);
...
move(robo1, 3, 2);    // move(robo1, 3, 2, 0);
```

1. 서브프로그램 개념

인수전달 고려 사항

가변인수 서브프로그램

- 받을 수 있는 인수 개수가 여러 가지가 될 수 있는 서브프로그램
- C, Java, Scheme 등

```
...
 static int sum(int ...ns) {
   int s = 0;
   for (int n: ns)  s += n;
   return s;
 }
 public static void main(String args[]) {
   PrintStream ps = new PrintStream(System.out);
   ps.println(sum(1));
   ps.println(sum(1, 2));
   ...
```

02

매개변수와 인수 전달

2. 매개변수와 인수 전달

인수 전달 모델

의미적 모델

- 프로그래머가 어떤 용도로 인수를 사용하느냐 하는 관점에서 바라본 모델

개념적 모델

- 자료 이동 측면에서 인수 전달 방식이 도식적으로 어떤 방식인지에 관한 모델

구현 모델

- 구현 관점에서 프로그래밍 언어가 인수 전달을 어떻게 구현하는지에 따른 모델

2. 매개변수와 인수 전달

- 의미적 모델
  - 입력 모드
    - 호출자의 실인수가 피호출자의 형식인수로 전달
    - 현재 대부분의 프로그래밍 언어의 기본
  - 출력 모드
    - 피호출자의 형식인수가 호출자의 실인수로 전달
  - 입출력 모드
    - 호출자와 피호출자 사이의 양방향 전달이 모두 발생

2. 매개변수와 인수 전달

- 개념적 모델
  - 값 전달
    - 실인수의 값을 물리적으로 복사하여 전달
  - 참조 전달
    - 실인수의 값을 참조할 수 있는 참조 경로를 전달

2. 매개변수와 인수 전달

구현 모델

- 값 전달(pass-by-value)
  - 실인수 값을 형식인수에 복사하여 전달
- 결과 전달(pass-by-result)
  - 형식인수의 값을 실인수에 복사하여 전달
- 값-결과 전달(pass-by-value-result)
  - 호출 시 실인수 값을 형식인수에 복사하여 전달하고
    복귀 시 형식인수 값을 실인수에 복사하여 전달
- 참조 전달(pass-by-reference)
  - 실인수에 대한 참조 경로 전달
- 이름 전달(pass-by-name)
  - 실인수 이름을 전달

2. 매개변수와 인수 전달

- 값 전달(pass-by-value)
  - 실인수 값을 형식인수에 복사하여 전달
  - 입력 모드의 인수 전달을 구현
  - 대부분의 언어에서 채택

구현 방법 1

- 데이터를 물리적으로 복사
- 인수 전달 효율성이 좋지 못함(예: 큰 배열)

```
int elem(Matrix m, int row, int col);
```

2. 매개변수와 인수 전달

- 값 전달(pass-by-value)
  - 실인수 값을 형식인수에 복사하여 전달
  - 입력 모드의 인수 전달을 구현
  - 대부분의 언어에서 채택

구현 방법 2

- 참조 경로 전달 및 쓰기 금지
- 간접 참조에 따른 시간 부담

```
int elem(const Matrix *mp, int row, int col);
```

2. 매개변수와 인수 전달

결과 전달(pass-by-result)

- 형식인수 값을 실인수에 복사하여 전달
- 출력 모드의 인수 전달을 구현

구현 방법 1

- 데이터를 물리적으로 복사
- 인수 전달 효율성이 좋지 못함
- 복사 순서에 따라 결과가 달라질 수 있음

```
void lor(int left, int right)
{ left = 3; right = 7; }
...
lor(a, a);
```

2. 매개변수와 인수 전달

결과 전달(pass-by-result)

- 형식인수 값을 실인수에 복사하여 전달
- 출력 모드의 인수 전달을 구현

구현 방법2

- 참조 경로 전달 및 실인수의 초깃값 사용 금지
- 실인수의 주소 값 결정 시기: 대부분 호출 시 결정

```
void lor(int left, int right)
{ left = 3; right = 7; }
...
idx = 0;
lor(idx, list[idx]);
```

2. 매개변수와 인수 전달

- 값-결과 전달(pass-by-value-result)
  - 호출 시 실인수 값을 형식인수로 복사
  - 복귀 시 형식인수 값을 실인수로 복사
  - 입출력 모드의 인수 전달을 구현

구현 방법

- 두 번 모두 물리적으로 값을 복사
- 복사에 의한 전달(pass-by-copy)

- 장점: 별칭 문제가 발생하지 않음
- 단점: 비효율을 비롯한 값 전달과 결과 전달의 단점이 모두 포함됨

2. 매개변수와 인수 전달

- 참조 전달(pass-by-reference)
  - 실인수의 주소(혹은 참조)를 형식인수에 전달
    - 실인수와 형식인수가 같은 변수
  - 입출력 모드의 인수 전달을 구현
  - 공유에 의한 전달(pass-by-sharing)
  - 장점: 전달 과정 자체가 효율적
  - 단점: 별칭(alias) 문제가 발생할 수 있음

2. 매개변수와 인수 전달

참조 전달(pass-by-reference)

별칭 문제

- 한 대상을 다른 이름으로 지칭하여 발생하는 문제
- 발생 상황
  - 실인수 충돌
  - 배열 원소 충돌:
    i==j인 경우 c=sqSum(a[i], a[j]);
  - 형식인수와 전역변수 충돌:
    전역변수를 사용하는 서브프로그램에 해당 전역변수를 실인수로 사용

```
#include <cstdio>

int sqSum(int &x1, int &x2)
{
  x1 = x1 * x1;
  x2 = x2 * x2;
  return x1 + x2;
}

int main()
{
  int a = 2;
  int b = 3;
  int c = sqSum(a, a);
  printf("%d\n", c);
}
```

2. 매개변수와 인수 전달

이름 전달(pass-by-name)

- 실인수 이름을 전달
  - 형식인수 사용 시 실인수 이름이 사용됨
- 특정 모델에 대응하지 않음

```
procedure double(x);
  integer x;
begin
  x := 2 * x
end;
```

```
i := 1;
double(i);
a[i] := 5;
double(a[i]);
```



```
i := 1;
i := 2 * i;
a[i] := 5;
a[i] := 2 * a[i];
```

2. 매개변수와 인수 전달

- 이름 전달(pass-by-name)
  - 장점
    - 늦은 바인딩으로 인한 유연함
    - 젠슨의 기법(Jensen's Device)과 같은 범용 서브프로그램 작성 가능
  - 단점
    - 인수 참조가 매우 비효율적
    - 인수 전달 의미가 난해하여 이해하기 어려움

```
procedure swap(a, b);
  integer a, b, temp;
begin
  temp := a;
  a := b;
  b := temp
end;
```

```
i := 1;
x[i] := 5;
swap(i, x[i]);
```



```
i := 1;
x[i] := 5;
temp := i;
i := x[i];
x[i] := temp;
```

2. 매개변수와 인수 전달

인수 전달 예(특정 언어와 무관)

- 참조 전달 vs 이름 전달

```
void swap(int a, int b) {
  int temp = a;
  a = b;
  b = temp;
}

int main() {
  int list[3] = {1,2,3};
  int i = 1;
  swap(i, list[i]);
  return 0;
}
```

		참조 전달	이름 전달
1	i	2	2
1	list[0]	1	1
2	list[1]	1	2
3	list[2]	3	1

2. 매개변수와 인수 전달

인수 전달 예(특정 언어와 무관)

- 참조 전달 vs 값-결과 전달

```
int i = 1;

void fun(int a, int b) {
  i = b;
}

int main() {
  int list[3] = {1, 2, 3};
  fun(i, list[i]);
  return 0;
}
```

		참조 전달	값-결과 전달
1	i	2	1
1	list[0]	1	1
2	list[1]	2	2
3	list[2]	3	3

03

서브프로그램 관련 주제

3. 서브프로그램 관련 주제

- 범용 서브프로그램
  - 여러 용도로 사용할 수 있는 서브프로그램
  - 다형 서브프로그램
    - 여러 타입의 인수에 적용 가능한 범용 서브프로그램
  - 다목적 서브프로그램
    - 다양한 목적에 적용 가능한 범용 서브프로그램

3. 서브프로그램 관련 주제

다형 서브프로그램

- 여러 타입의 인수에 적용 가능한 서브프로그램
- 제너릭(generic) 서브프로그램
- C 언어: 매크로 확장과 void * 타입을 이용
  - 인수 타입 검사 불가

```
#define MIN(A, B)  ((A) < (B) ? (A) : (B))
```

- C++는 템플릿, Java는 제너릭 이용
  - 매개변수적 다형성: 타입 인수를 받아 여러 타입을 다룸

```
sort(a, a+nelem, less<int>());
```

3. 서브프로그램 관련 주제

- 다목적 서브프로그램
  - 다양한 목적에 적용 가능한 서브프로그램
  - 젠슨의 기법(Jensen's Device)
    - 이름 전달의 늦은 바인딩을 이용
  - Algol 60

```
begin
  real procedure sum(i, lo, hi, term);
    value lo, hi;
    integer i, lo, hi;
    real term;
  begin
    real temp;
    temp := 0;
    for i := lo step 1 until hi do
      temp := temp + term;
    sum := temp
  end;
  print(sum(i, 1, 10, i * i))
end
```

3. 서브프로그램 관련 주제

분리 컴파일과 독립 컴파일

- 각 모듈을 다른 프로그래머가 작성
  - 독자적으로 자신의 소스코드를 컴파일할 수 있어야 함

분리 컴파일

- 코드 사이의 인터페이스 정보를 이용하여 프로그램 일부를 다른 부분과 분리하여 컴파일하는 방법
- Ada, Fortran 90, Modula-2 등

독립 컴파일

- 인터페이스 정보 없이 프로그램 일부를 다른 부분과 분리하여 컴파일하는 방법
- Fortran II~77, C, C++ 등

정리하기

- 서브프로그램은 독자적인 입력과 출력을 갖춘 프로그램 조각으로 모듈화 프로그래밍을 지원하기 위해 기본적으로 필요한 기능임
- 서브프로그램은 반환값의 유무에 따라 프로시저와 함수로 분류할 수 있음
- 서브프로그램의 매개변수 프로파일과 함수 반환 타입을 합쳐 서브프로그램 프로토콜이라고 부름
- 외부로부터 전달될 정보를 받기 위해 서브프로그램 정의 부분에서 선언한 변수를 매개변수라 하고, 서브프로그램 호출 시 매개변수로 전달될 의도로 사용된 수식을 인수라고 함
- 인수 전달의 구현 모델로는 값 전달, 결과 전달, 값-결과 전달, 참조 전달, 이름 전달이 있음
- 여러 용도로 사용할 수 있는 서브프로그램을 범용 서브프로그램이라고 하며 이는 다형 서브프로그램과 다목적 서브프로그램으로 구성됨

14강

다음시간안내 ▶▶▶

서브프로그램 구현